L27 — Queues: Introduction and Operations

Unit: 2 | **Chapter:** Queues | **BT Level:** BT3 | **CO:** CO2

Learning Objectives

- Explain the FIFO principle and Queue ADT
- Implement enqueue and dequeue using arrays (circular queue)
- Implement queue using a linked list
- Analyze time complexity of all queue operations

---

1. Queue Concept (FIFO)

A **queue** is a linear data structure that follows the **First In, First Out (FIFO)** principle.

- Elements are added at the **rear** (back)
- Elements are removed from the **front**

```
Enqueue  →  [  front  →  →  →  rear  ]  →  Dequeue
             item1  item2  item3
```

Real-world analogies:

- Waiting line at a ticket counter
- Print job queue in an OS
- Network packet buffers
- BFS traversal queue

---

2. Queue ADT — Operations

Operation	Description	Time
`enqueue(item)`	Add item to rear	O(1)
`dequeue()`	Remove and return item from front	O(1)
`peek()` / `front()`	View front item without removing	O(1)
`is_empty()`	Check if queue has no elements	O(1)
`size()`	Return number of elements	O(1)

---

3. Array-Based Queue (Naive)

```
class ArrayQueue:
    def __init__(self, capacity=100):
        self.data = [None] * capacity
```

```
        self.capacity = capacity
        self.front_idx = 0
        self.rear_idx = -1
        self._size = 0

    def enqueue(self, item):
        if self._size == self.capacity:
            raise OverflowError("Queue is full")
        self.rear_idx += 1
        self.data[self.rear_idx] = item
        self._size += 1

    def dequeue(self):
        if self.is_empty():
            raise IndexError("Queue is empty")
        item = self.data[self.front_idx]
        self.front_idx += 1
        self._size -= 1
        return item
```

Problem: `front_idx` keeps increasing — wasted space at the front.
Solution: Circular Array Queue

---

4. Circular Array Queue

Wrap around using modulo arithmetic: index `% capacity`

```
Capacity = 5, after several enqueue/dequeue cycles:
  Index:  0    1    2    3    4
          [20] [30] [ ]  [10] [ ]
           ↑                  ↑
          rear               front (front=3, rear=1)
```

```
class CircularQueue:
    def __init__(self, capacity=100):
        self.data = [None] * capacity
        self.capacity = capacity
        self.front = 0
        self.rear = 0     # Points to next empty slot
        self._size = 0

    def is_empty(self):
        return self._size == 0

    def is_full(self):
        return self._size == self.capacity

    def enqueue(self, item):
        if self.is_full():
            raise OverflowError("Queue is full")
        self.data[self.rear] = item
        self.rear = (self.rear + 1) % self.capacity
```

```
        self._size += 1

    def dequeue(self):
        if self.is_empty():
            raise IndexError("Queue is empty")
        item = self.data[self.front]
        self.data[self.front] = None    # Optional: clear slot
        self.front = (self.front + 1) % self.capacity
        self._size -= 1
        return item

    def peek(self):
        if self.is_empty():
            raise IndexError("Queue is empty")
        return self.data[self.front]

    def size(self):
        return self._size
```

Circular Queue Trace (capacity = 5)

Operation	front	rear	size	Array
Initial	0	0	0	[_, _, _, _, _]
enqueue(10)	0	1	1	[10, _, _, _, _]
enqueue(20)	0	2	2	[10, 20, _, _, _]
enqueue(30)	0	3	3	[10, 20, 30, _, _]
dequeue() → 10	1	3	2	[_, 20, 30, _, _]
enqueue(40)	1	4	3	[_, 20, 30, 40, _]
enqueue(50)	1	0	4	[_, 20, 30, 40, 50]
enqueue(60)	1	1	5	[60, 20, 30, 40, 50]
dequeue() → 20	2	1	4	[60, _, 30, 40, 50]

---

5. Linked List Queue

Avoids fixed capacity — dynamic size.

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class LinkedQueue:
    def __init__(self):
        self.front = None   # Remove from here
        self.rear = None    # Add to here
        self._size = 0

    def is_empty(self):
        return self.front is None
```

```
    def enqueue(self, item):
        new_node = Node(item)
        if self.rear is None:
            self.front = self.rear = new_node
        else:
            self.rear.next = new_node
            self.rear = new_node
        self._size += 1

    def dequeue(self):
        if self.is_empty():
            raise IndexError("Queue is empty")
        item = self.front.data
        self.front = self.front.next
        if self.front is None:     # Queue became empty
            self.rear = None
        self._size -= 1
        return item

    def peek(self):
        if self.is_empty():
            raise IndexError("Queue is empty")
        return self.front.data
```

Linked Queue Trace

```
Initial: front=None, rear=None

enqueue(10): front → [10|None] ← rear
enqueue(20): front → [10|→] → [20|None] ← rear
enqueue(30): front → [10|→] → [20|→] → [30|None] ← rear
dequeue() → 10: front → [20|→] → [30|None] ← rear
dequeue() → 20: front → [30|None] ← rear
```

---

6. Python's Built-in Queue Support

```
from collections import deque

q = deque()
q.append(10)       # enqueue to right
q.append(20)
q.append(30)
print(q.popleft()) # dequeue from left → 10
print(q[0])        # peek → 20

# deque is O(1) for both ends — backed by doubly-linked list
```

---

7. Comparison Table

Feature	Circular Array Queue	Linked List Queue
Size	Fixed	Dynamic
Memory	Preallocated	Allocated per node
Enqueue	O(1)	O(1)
Dequeue	O(1)	O(1)
Peek	O(1)	O(1)
Overflow	Possible	No (only memory limit)
Cache performance	Better (contiguous)	Worse (pointer chasing)

Summary

- Queue: **FIFO** — elements added at rear, removed from front.
- **Circular array** solves the wasted-space problem of naive array queues.
- **Linked list queue** is dynamically sized with O(1) enqueue/dequeue using front/rear pointers.
- Python: use `collections.deque` for efficient O(1) queue operations.

References

- Cormen et al., *Introduction to Algorithms*, Ch. 10.1 (MIT Press, 2022)
- Skiena, *The Algorithm Design Manual*, Ch. 3.2 (Springer, 2020)